

ARC-AGI 1 Architecture Design Report

Team_12

1 Our Approach

1.1 Architecture

We propose a **Recursive Transformer** architecture designed to model iterative reasoning processes required for ARC tasks. The model is trained entirely from scratch and operates comfortably within the 50M parameter constraint. The backbone configuration consists of:

- 6 transformer layers
- hidden dimension $D_{model} = 256$
- 8 attention heads
- iterative refinement depth $T = 5$

Rather than producing predictions in a single forward pass, the model performs multiple refinement iterations. At each iteration, the transformer updates an implicit latent reasoning representation which gradually improves the predicted output grid. This recursive formulation allows intermediate representations to encode partial hypotheses about spatial transformations.

The choice of $T = 5$ reflects an engineering tradeoff between reasoning depth and computational cost. Empirically, additional iterations beyond five provided diminishing returns while increasing training time and GPU memory usage. Iterative refinement also stabilizes learning by allowing attention layers to progressively correct earlier prediction errors.

Transformer attention is particularly well-suited for ARC because it enables relational reasoning across spatial locations. Self-attention allows each grid cell to incorporate contextual information from all other cells, enabling the model to capture global spatial patterns such as symmetry, repetition, and object alignment.

1.2 Representation

Each ARC grid has maximum resolution 30×30 . Every grid cell is converted into a token represented by a 10-dimensional one-hot encoding corresponding to colour identity. Colours in ARC are categorical symbols rather than numeric values, therefore one-hot encoding prevents the model from learning artificial ordinal relationships between colour indices.

All grids are padded to 30×30 to allow batching across tasks with varying sizes. Padding masks ensure that padded cells do not contribute to the loss function.

This token-based representation allows the transformer to process ARC tasks as structured symbolic reasoning problems rather than continuous-valued image tasks.

1.3 Handling Multiple Demonstration Pairs

Each ARC task contains between two and four demonstration input-output pairs. These demonstrations are concatenated along the token dimension, enabling the model to jointly condition on multiple examples when inferring transformation rules.

Pair-index embeddings allow the transformer to distinguish which tokens correspond to which demonstration pair. This is essential because ARC reasoning often requires identifying consistent transformation patterns across examples. By attending across demonstration pairs, the model can compare structural relationships between input and output grids.

This joint conditioning mechanism enables meta-learning behaviour, where the model infers task-specific rules directly from demonstrations without explicit parameter updates.

1.4 Positional Encoding

Spatial structure plays a central role in ARC tasks. Flattening a 2D grid into a 1D sequence removes geometric relationships between cells. To preserve spatial structure, we use structured 3D positional embeddings consisting of learned embeddings for:

- row index
- column index
- demonstration pair index

Row and column embeddings preserve grid topology, while pair-index embeddings distinguish demonstration context. This allows the model to represent both spatial and relational structure simultaneously.

3D positional encoding is particularly useful for compositional generalization. Many ARC tasks require applying the same transformation across multiple objects or spatial regions. Explicit spatial encoding allows attention layers to identify repeating structural motifs.

1.5 Prediction Strategy

The model predicts the entire output grid simultaneously using parallel cell prediction. A linear projection maps token embeddings to probability distributions over 10 colour classes, followed by softmax applied independently at each grid cell.

Autoregressive decoding was avoided because sequential prediction would require up to 900 steps for a single grid, increasing computational cost significantly. Parallel prediction improves efficiency while allowing dependencies between cells to be modeled through attention.

Iterative refinement compensates for the lack of autoregressive structure by allowing predictions to be progressively updated across refinement steps.

1.6 Loss Function

Training uses token-level cross entropy loss applied across valid grid cells. Deep supervision is used by computing loss at each refinement iteration:

$$\mathcal{L} = \sum_{t=1}^T \text{CE}(y_t, y^*)$$

Deep supervision improves gradient flow through the network and prevents early refinement steps from learning degenerate representations. This is particularly important in iterative architectures where gradients must propagate through repeated transformer layers.

1.7 Parameter Count

The model contains 4,823,306 parameters ($\approx 4.8\text{M}$), significantly below the 50M parameter limit. Parameter distribution is summarized below:

Component	Parameters
Transformer backbone	$\sim 3.9\text{M}$
Embedding layers	$\sim 0.7\text{M}$
Output head	$\sim 0.2\text{M}$

Maintaining a compact parameter count helps prevent overfitting given the small dataset size.

1.8 Training Strategy

Optimization uses AdamW with weight decay regularization. Because ARC contains only 400 training tasks, augmentation is necessary to improve generalization.

We apply dihedral transformations:

- rotations
- horizontal flips
- vertical flips

These transformations preserve semantic structure while increasing spatial diversity.

We also apply controlled probabilistic colour permutation. Fully random permutation was avoided because some tasks rely on absolute colour identity rather than relative colour relationships.

During evaluation, Test-Time Training (TTT) is used with 50 gradient steps per task at learning rate 3×10^{-4} . TTT allows the model to adapt to task-specific distributions using demonstration pairs. This improves performance on tasks with distribution shifts not captured during global training.

2 Ablations

We conducted controlled ablation experiments to quantify the contribution of key architectural components. Each variant was trained for 15 epochs using identical optimization settings to ensure fair comparison. The full model achieved 5.41% exact-match accuracy on the validation set and serves as the reference configuration.

Design Choice	Proposed	Baseline	Gain
Loss Strategy	Deep supervision	Final-step loss	+2.70%
Positional Encoding	3D positional	1D sinusoidal	+5.41%
Refinement Depth	$T = 5$ iterations	$T = 1$ pass	+1.35%

Table 1: Ablation results on validation tasks

Effect of Deep Supervision. Removing intermediate supervision reduced accuracy from 5.41% to 2.70%. When loss is applied only at the final refinement step, early iterations receive weak gradient signals, resulting in unstable intermediate representations. Deep supervision ensures that every refinement stage contributes meaningfully to the learning process, improving convergence stability.

Effect of Spatial Positional Encoding. Replacing structured 3D positional embeddings with 1D sinusoidal encoding reduced performance to 0%. Flattening the grid removes explicit

spatial relationships between cells, making it difficult for the model to learn geometric transformations such as reflection, translation, and symmetry. This confirms that preserving spatial topology is essential for ARC reasoning.

Effect of Iterative Refinement. Reducing refinement depth from $T = 5$ to $T = 1$ decreased accuracy from 5.41% to 4.05%. Single-pass prediction forces the model to infer the full transformation rule immediately, whereas iterative refinement allows progressive correction of intermediate predictions. This iterative reasoning process improves the model’s ability to capture multi-step transformations.

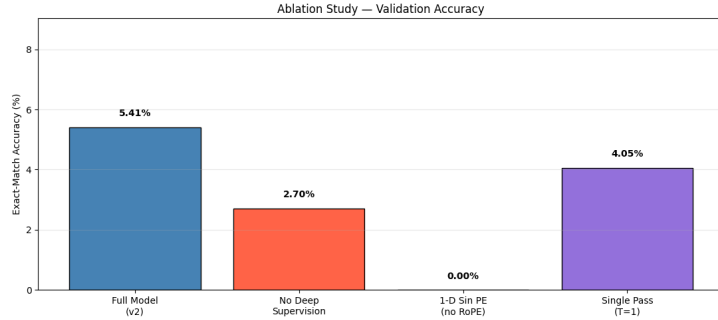


Figure 1: Validation accuracy comparison across ablation settings

3 Results

Evaluation was performed on 400 ARC evaluation tasks using two inference attempts per task with Test-Time Training (TTT) applied for 50 gradient steps per task. The final exact-match accuracy was:

3.25% (13 / 400 tasks)

```

Evaluating 400 tasks (TTT = 50 steps each)...
  Task | Running Acc | Elapsed | ETA
  -----
    50 |    2.00% |    382s | 2677s
   100 |    2.00% |    759s | 2277s
   150 |    1.33% |   1149s | 1914s
   200 |    1.50% |   1550s | 1550s
   250 |    2.00% |   1935s | 1161s
   300 |    2.00% |   2315s |   772s
   350 |    2.29% |   2703s |   386s
   400 |    3.25% |   3073s |     0s

=====
Final Evaluation Exact-Match Accuracy: 13/400 (3.25%)
=====

```

Figure 2: Evaluation progress

The model demonstrates strongest performance on tasks involving geometric regularities such as symmetry detection, spatial transformations, pattern completion, and colour propagation rules. These categories align well with the architectural inductive bias introduced by structured 3D positional embeddings and dihedral augmentation, which preserve spatial relationships across grid cells. Transformer self-attention enables global interaction between distant cells, allowing the model to identify repeated structures and consistent geometric mappings

across demonstrations. In contrast, performance remains limited on tasks requiring counting, conditional rule composition, or multi-stage logical reasoning. These problems typically require discrete symbolic manipulation or persistent intermediate memory, which are difficult to approximate using continuous attention representations with limited refinement depth. Despite applying 50-step Test-Time Training adaptation for each task, improvements were incremental, suggesting that the primary limitation lies in representational capacity for algorithmic reasoning rather than optimization alone.

4 What Did Not Work

During development, multiple alternative architectural and training strategies were explored. Although these approaches did not improve final accuracy, they provided insight into the challenges of learning algorithmic reasoning from limited supervision.

4.1 Unconstrained Colour Permutation

We initially applied fully random colour permutation as an augmentation strategy to increase dataset diversity. The intuition was that colour shuffling would encourage the model to focus on structural transformations rather than memorizing colour mappings.

However, many ARC tasks depend on absolute colour identity rather than purely relational structure. Random permutation disrupted this semantic consistency, producing contradictory supervision signals. For example, tasks that require identifying a specific colour (e.g., “expand all blue regions adjacent to red objects”) became ambiguous after permutation.

Replacing unrestricted permutation with controlled probabilistic permutation preserved semantic structure while still improving generalization.

4.2 CNN-based Spatial Architectures

We evaluated CNN-based models to capture local spatial patterns efficiently. Convolutional layers successfully learned local texture patterns but struggled to model long-range dependencies between distant cells.

Many ARC tasks require reasoning about relationships between spatially separated objects, such as symmetry detection or alignment of distant components. CNN receptive fields grow gradually with depth, making it difficult to capture global structure without significantly increasing model size.

These experiments motivated the use of transformer attention, which allows direct interaction between all spatial positions.

4.3 Single-pass Transformer

We tested transformer variants that produced predictions in a single forward pass ($T = 1$). These models converged faster but achieved lower exact-match accuracy.

Without iterative refinement, early prediction errors cannot be corrected in later stages. Recursive passes allow the model to progressively refine hypotheses, improving performance on tasks requiring multi-step reasoning.

This suggests that iterative reasoning depth is more beneficial than simply increasing model capacity.

4.4 1D Positional Encoding

Replacing structured spatial embeddings with standard 1D sinusoidal encoding resulted in near-zero accuracy. Flattening the grid removes explicit row-column relationships, forcing the model to infer spatial structure implicitly through attention.

This significantly increases learning difficulty and prevents reliable generalization of spatial transformations. The result highlights the importance of incorporating spatial inductive bias directly into the representation.

4.5 Increasing Model Size

Increasing hidden dimension or number of transformer layers did not produce consistent performance improvements. Larger models showed signs of overfitting due to the small dataset size (400 tasks).

Additionally, deeper models increased memory consumption and training instability on Colab T4 hardware. These observations suggest that architectural inductive bias is more important than raw parameter count for ARC tasks.

4.6 Sensitivity of Test-Time Training

Test-Time Training improves performance by adapting model parameters using demonstration pairs. However, performance was sensitive to the number of gradient steps.

Too few adaptation steps led to underfitting task-specific patterns, while excessive adaptation caused overfitting to demonstration noise. Empirically, 50 gradient steps with learning rate 3×10^{-4} provided the best trade-off.

This indicates that adaptation dynamics are a critical factor in few-shot reasoning problems.

4.7 Observed Failure Patterns

Across experiments, consistent failure modes appeared in tasks requiring:

- counting objects with varying cardinality
- applying transformations conditioned on object size
- multi-stage logical reasoning
- persistent symbolic memory

These tasks require discrete reasoning steps rather than purely continuous pattern recognition. Standard transformer attention approximates reasoning through continuous representations, which may be insufficient for exact symbolic manipulation.

Future improvements may require hybrid architectures combining neural perception with explicit symbolic reasoning mechanisms.